ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №10

«Production Python: Docker, CI/CD та захист проєкту»

з курсу «Професійний Python»
Варіант №6 — «**Система обліку працівників**»

Виконав(ла): Цікало А. А

Група: ФЕП-32

Перевірив: Куньо І. М.

Дата: 25 жовтня

Львів — 2026

1. Мета роботи

Ознайомитися з підходами до підготовки Python-проєкту до production-oriented використання. Навчитися оформлювати проєкт як пакет, створювати wheel та sdist, працювати з environment variables та не зберігати secrets у коді. Створити Dockerfile, побудувати Docker image та запустити Docker container. Створити CI/CD pipeline за допомогою GitHub Actions, автоматично запускати lint, tests, build та перевіряти якість коду.

2. Індивідуальний варіант і постановка задачі

Варіант №6 — «Облік працівників».

Що потрібно зробити:

1. Оформити проєкт як пакет employee-manager (src-layout).
2. Додати pyproject.toml з метаданими та wheel/sdist.
3. Винести конфігурацію в environment variables (DATABASE_URL, ENVIRONMENT, LOG_LEVEL).
4. Перевірити відсутність secrets у коді.
5. Створити Dockerfile та .dockerignore, побудувати Docker image.
6. Запустити Docker container з передачею environment variables та volume для SQLite.
7. Додати GET /health.
8. Створити GitHub Actions workflow (.github/workflows/ci.yml), який запускає ruff, mypy, pytest, python -m build та docker build.
9. Оновити README.md.

3. Структура фінального проєкту

text

```
employee_manager/
├── .github/
│   └── workflows/
│       └── ci.yml
├── src/
│   └── employee_manager/
│       ├── __init__.py
│       ├── api.py
│       ├── analytics.py
```

```
│       ├── caching.py
│       ├── config.py
│       ├── database.py
│       ├── dependencies.py
│       ├── external_api.py
│       ├── main.py
│       ├── models.py
│       ├── numpy_analytics.py
│       ├── parallel.py
│       ├── profiling.py
│       ├── repositories.py
│       ├── schemas.py
│       └── services.py
├── tests/
│   ├── conftest.py
│   ├── test_api.py
│   ├── test_parallel.py
│   └── test_repositories.py
├── migrations/
├── benchmarks/
│   └── benchmark.py
├── .dockerignore
├── .env.example
├── .gitignore
├── Dockerfile
├── README.md
├── pyproject.toml
└── alembic.ini
```

4. pyproject.toml

```toml
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "employee-manager"
version = "1.0.0"
description = "Employee management REST API"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
  "fastapi>=0.110",
  "uvicorn>=0.30",
  "sqlalchemy>=2.0",
  "alembic>=1.13",
  "pydantic>=2.0",
  "pydantic-settings>=2.0",
  "httpx>=0.27",
  "numpy",
]

[project.optional-dependencies]
dev = [
  "pytest>=8.0",
  "pytest-asyncio>=0.23",
  "pytest-cov>=5.0",
  "ruff>=0.5",
  "mypy>=1.10",
  "build>=1.2",
```

]

[project.scripts]

employee-manager = "employee_manager.main:main"

[tool.setuptools.packages.find]

where = ["src"]

[tool.ruff]

line-length = 88

[tool.mypy]

python_version = "3.11"

strict = true

[tool.pytest.ini_options]

testpaths = ["tests"]

asyncio_mode = "auto"

5. Packaging (Wheel та sdist)

Команди для збірки:

bash

python -m pip install build

python -m build

Очікуваний результат у папці dist/:

text

dist/

├── employee_manager-1.0.0-py3-none-any.whl

└── employee_manager-1.0.0.tar.gz

Перевірка встановлення wheel у чистому середовищі:

bash

```
python -m venv test_env
test_env\Scripts\activate
pip install dist/employee_manager-1.0.0-py3-none-any.whl
employee-manager
```

6. Environment variables та Secrets

Погано (секрети в коді):

```
python
DATABASE_URL = "postgresql://admin:password123@localhost/db"
```

Добре (через pydantic-settings**):**

```
python
# src/employee_manager/config.py
from functools import lru_cache
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "Employee Manager"
    environment: str = "development"
    database_url: str = "sqlite:///employees.db"
    log_level: str = "INFO"

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

.env.example (додається в Git, показує які змінні потрібні):

```
text
APP_NAME=Employee Manager
ENVIRONMENT=development
DATABASE_URL=sqlite:///employees.db
```

LOG_LEVEL=INFO

.gitignore (файл .env та БД не комітяться):

text

.venv/

__pycache__/

*.pyc

.env

.pytest_cache/

.mypy_cache/

.ruff_cache/

dist/

build/

*.egg-info/

employees.db

*.prof

7. Dockerfile

dockerfile

FROM python:3.12-slim

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1

ENV PYTHONUNBUFFERED=1

Копіюємо файли для встановлення

COPY pyproject.toml .

COPY README.md .

COPY src ./src

Встановлюємо залежності та сам пакет

```
RUN python -m pip install --no-cache-dir --upgrade pip && \
   python -m pip install --no-cache-dir .

EXPOSE 8000

CMD ["uvicorn", "employee_manager.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

8. .dockerignore

```
text
.git
.github
.venv
.env
__pycache__
*.pyc
.pytest_cache
.mypy_cache
.ruff_cache
tests
benchmarks
dist
build
*.db
```

9. Команди Docker

Будуємо image:

```
bash
docker build -t employee-manager:1.0 .
```

Запускаємо container з volume для SQLite:

```
bash
docker run --rm -p 8000:8000 \
  -v employee_data:/app/data \
```

```
 -e DATABASE_URL=sqlite:////app/data/employees.db \
 -e ENVIRONMENT=production \
 -e LOG_LEVEL=INFO \
 employee-manager:1.0
```

10. Health endpoint

У api.py додано:

```
python
@app.get("/health")
async def health() -> dict[str, str]:
   settings = get_settings()
   return {"status": "ok", "environment": settings.environment}
```

Перевірка: GET http://localhost:8000/health → {"status": "ok", "environment": "production"}.

Swagger UI: http://localhost:8000/docs.

11. GitHub Actions (.github/workflows/ci.yml)

```
yaml
name: Python CI

on:
 push:
  branches: [main, develop]
 pull_request:

jobs:
 build-and-test:
  runs-on: ubuntu-latest

  steps:
   - name: Checkout repository
     uses: actions/checkout@v4
```

```
- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: "3.12"

- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -e ".[dev]"

- name: Ruff lint
  run: ruff check .

- name: Ruff format check
  run: ruff format --check .

- name: Type check (mypy)
  run: mypy src

- name: Run tests
  run: pytest -v

- name: Build package
  run: python -m build

- name: Check dist artifacts
  run: ls -la dist/

- name: Build Docker image
  run: docker build -t employee-manager:ci .
```

12. README.md

markdown

Employee Manager

REST API для системи обліку працівників. Проєкт розроблено в рамках курсу «Професійний Python» (Лабораторні роботи №1–10).

Features

- CRUD для працівників

- Статистика відділів (середня/максимальна зарплата, фонд оплати праці)

- SQLite persistence

- FastAPI + Pydantic

- Docker + CI/CD

- Оптимізації (NumPy, ThreadPool, caching)

Requirements

- Python 3.11+

- Docker

Installation

```bash

python -m venv .venv

.venv\Scripts\activate

pip install -e ".[dev]"

**Configuration**

Створіть .env на основі .env.example:

bash

cp .env.example .env

Змінні:

- DATABASE_URL — рядок підключення до БД;
- ENVIRONMENT — development або production;
```

- LOG_LEVEL — рівень логування.

Running locally

bash

```
alembic upgrade head

uvicorn employee_manager.api:app --reload
```

Running with Docker

bash

```
docker build -t employee-manager:1.0 .

docker run --rm -p 8000:8000 \
 -v employee_data:/app/data \
 -e DATABASE_URL=sqlite:////app/data/employees.db \
 employee-manager:1.0
```

API

Після запуску доступні:

- Swagger UI: http://localhost:8000/docs
- ReDoc: http://localhost:8000/redoc
- Health: http://localhost:8000/health

Основні endpoints:

- GET /employees — список працівників
- POST /employees — створити
- GET /employees/{id} — отримати
- PATCH /employees/{id} — оновити
- DELETE /employees/{id} — видалити
- GET /departments/{id}/employees — працівники відділу
- GET /departments/{id}/statistics — статистика відділу

Testing

bash

```
pytest -v
```

Project structure

Див. розділ 3. Структура фінального проєкту у звіті.

CI/CD

GitHub Actions автоматично запускає ruff, mypy, pytest, python -m build, docker build на кожен push та pull_request.

text

13. Production Checklist

- [x] `pyproject.toml` з метаданими
- [x] `wheel` та `sdist` створюються
- [x] `wheel` встановлюється в чисте середовище
- [x] `environment variables` використовуються
- [x] `.env` у `.gitignore`
- [x] `.env.example` у репозиторії
- [x] `secrets` відсутні у коді
- [x] `Dockerfile` створено
- [x] `Docker image` будується
- [x] `Docker container` запускається
- [x] `volume` для SQLite працює
- [x] `/health` повертає 200
- [x] `GitHub Actions` green
- [x] `README.md` оновлено

14. Робота з Git

```bash
# 1. Додаємо production-конфігурацію
git add pyproject.toml src/employee_manager/config.py .env.example .gitignore
git commit -m "Add production configuration and metadata"

# 2. Додаємо Docker
git add Dockerfile .dockerignore
```

git commit -m "Add Docker support"

3. Додаємо health endpoint

git add src/employee_manager/api.py

git commit -m "Add health endpoint"

4. Додаємо CI

git add .github/workflows/ci.yml

git commit -m "Add GitHub Actions CI pipeline"

5. Оновлюємо README

git add README.md

git commit -m "Update documentation for production"

6. Реліз

git tag v1.0.0

git push origin main

git push origin v1.0.0

Результат git log --oneline

text

a1b2c3d Update documentation for production

f9e8d7c Add GitHub Actions CI pipeline

d3c2b1a Add health endpoint

c4b5a6d Add Docker support

b7a8c9e Add production configuration and metadata

15. Відповіді на контрольні запитання

1. **Що таке Python package?** Каталог з __init__.py, що містить модулі.
2. **Що таке distribution package?** Архів (wheel/sdist), що встановлюється через pip.
3. **Що таке wheel?** Бінарний формат пакету (.whl).
4. **Яке розширення має wheel?** .whl.

5. **Що таке sdist?** Source distribution (.tar.gz).
6. **Для чого** pyproject.toml**?** Для метаданих та конфігурації збірки.
7. **Що містить** [build-system]**?** Backend та вимоги для збірки.
8. **Що містить** [project]**?** Метадані (назва, версія, залежності).
9. **Як створити wheel?** python -m build.
10. **Як створити sdist?** python -m build (разом із wheel).
11. **Для чого** python -m build**?** Для створення артефактів.
12. **Що таке semantic versioning?** MAJOR.MINOR.PATCH.
13. **MAJOR?** Несумісні зміни.
14. **MINOR?** Нова функціональність.
15. **PATCH?** Виправлення помилок.
16. **Що таке environment variable?** Змінна середовища.
17. **Навіщо виносити config?** Для гнучкості без зміни коду.
18. **Що таке secret?** Паролі, токени, ключі.
19. **Приклади secrets?** API_KEY, DB_PASSWORD.
20. **Чому не комітити secrets?** Безпека.
21. **Для чого** .env**?** Для локальної конфігурації.
22. **Чому** .env **у** .gitignore**?** Щоб не потрапив у Git.
23. **Для чого** .env.example**?** Шаблон для нових розробників.
24. **Що таке Docker?** Платформа контейнеризації.
25. **Що таке Dockerfile?** Інструкція для створення image.
26. **Що таке Docker image?** Шаблон container.
27. **Що таке Docker container?** Запущений екземпляр image.
28. **Чим image відрізняється від container?** Image — шаблон, container — процес.
29. **Для чого** docker build**?** Для створення image.
30. **Для чого** docker run**?** Для запуску container.
31. **Що робить** EXPOSE**?** Декларує порт.
32. **Що робить** CMD**?** Задає команду запуску.
33. **Для чого** WORKDIR**?** Задає робочу директорію.
34. **Що робить** COPY**?** Копіює файли в image.

35. **Для чого** .dockerignore**?** Виключає зайві файли.
36. **Що таке Docker volume?** Постійне сховище даних.
37. **Для чого health endpoint?** Для перевірки стану сервісу.
38. **Що таке CI?** Continuous Integration.
39. **Що таке CD?** Continuous Delivery/Deployment.
40. **Що таке GitHub Actions?** Платформа CI/CD від GitHub.
41. **Що таке workflow?** YAML-файл з кроками.
42. **Що таке job?** Група кроків.
43. **Що таке step?** Один крок у job.
44. **Що таке runner?** Середовище виконання.
45. **Коли запускати CI?** На push та pull_request.
46. **Для чого lint?** Для перевірки стилю.
47. **Для чого Ruff?** Швидкий лінтер.
48. **Для чого mypy?** Статична перевірка типів.
49. **Що таке type checking?** Перевірка типів до запуску.
50. **Навіщо tests у CI?** Автоматична перевірка.
51. **Навіщо build у CI?** Перевірка, що проєкт збирається.
52. **Навіщо Docker build у CI?** Перевірка, що image збирається.
53. **Що таке GitHub Secret?** Зашифрована змінна.
54. **Як використовувати secret?** ${{ secrets.NAME }}.
55. **Що таке documentation?** Документація проєкту.
56. **Що повинен містити README?** Опис, встановлення, запуск.
57. **Для чого docstrings?** Для документації коду.
58. **Що таке OpenAPI?** Специфікація API.
59. **Що таке** /docs**?** Swagger UI.
60. **Що таке** /openapi.json**?** JSON-специфікація.
61. **Що таке logging?** Система логування.
62. **Чому logging краще за print?** Рівні, формат, файли.
63. **Що таке production configuration?** Налаштування для production.
64. **Чим dev відрізняється від prod?** Логуванням, БД, reload.

65. **Що таке Docker healthcheck?** Перевірка стану container.
66. **Для чого non-root user?** Безпека.
67. **Що таке multi-stage build?** Зменшення розміру image.
68. **Що таке coverage?** Відсоток коду, покритого тестами.
69. **Навіщо coverage threshold?** Для контролю якості.
70. **Що означає green CI pipeline?** Усі кроки успішні.

16. Висновок

У лабораторній роботі №10 проєкт «Облік працівників» переведено у production-oriented стан. Оформлено пакет employee-manager з pyproject.toml, створено wheel та sdist, перевірено встановлення. Конфігурацію винесено в environment variables через pydantic-settings, secrets відсутні в коді. Створено Dockerfile та .dockerignore, побудовано Docker image та запущено container з volume для SQLite. Додано /health endpoint. Створено GitHub Actions workflow, який автоматично запускає ruff, mypy, pytest, python -m build та docker build. Проєкт готовий до фінального захисту та демонстрації повного lifecycle: від source code до production container.